CS 61A — Structure and Interpretation of Computer Programs

Exam-Level Study Guide

Course: Computer Science 61A — Structure and Interpretation of Computer Programs

Languages: Python (primary), Scheme, SQL

Course site: https://cs61a.org/

Final exam: Tue May 12, 2026 — exact time/location: verify at https://cs61a.org/articles/about-61a/

Format: In-person, written. ~3 hours. Cheat sheet typically 2 double-sided handwritten pages.

How to use this guide. Each topic block follows the same shape:

- **Definition** — formal one-line statement.
- **Plain English** — what it actually means.
- **Canonical template** — the code you should be able to write from memory.
- **Variables / pieces** — what each part of the template means.
- **Trace example or diagram** — concrete worked example or environment sketch.
- **Exam patterns / pitfalls** — what 61A finals do with this concept.

Calibration note: Coverage is concept-level. For pattern-trigger references at the "see X → write Y" level, use your existing `61A_Exam_Reference.md` and `MT2_Code_Trigger_Sheet.md` as primary; this guide is the structured catalog over both.

Table of Contents

---

1. Expressions, Functions, and Control Flow

[Slides 02–03, 07]

Pure vs Non-Pure Functions

Definition. A pure function returns a value with no side effects. A non-pure function has effects (printing, mutation) and may return `None`.

Plain English. `abs(-5)` returns `5` — pure. `print(5)` *displays* `5` and returns `None` — non-pure. The most common trap is treating `print(...)` as if it returns the value it printed.

Canonical examples.

```
def square(x):        # pure — returns a value
    return x * x

def announce(x):      # non-pure — prints, returns None
    print(x)
```

Exam patterns / pitfalls.

- `print(print(1))` displays `1` then displays `None`. Returns `None`.
- `1 + print(2)` raises `TypeError` because `print` returns `None`.
- Always read the question for "*displayed*" vs "*returned*."

Function Definitions

Definition. `def name(params): body` creates a function object and binds the name in the current frame.

Plain English. `def` is itself a statement. It creates a function value. The function value remembers its parent frame (the frame in which it was defined).

Canonical template.

```
def f(x, y):        # signature: name + parameters
    z = x + y       # body: any statements
    return z        # return statement (without it, returns None)
```

Control Flow Essentials

`if/elif/else` — only one branch executes.

Boolean short-circuit — `and`/`or` return one of their operands, not necessarily `True`/`False`.

- `0 and 5` → `0` (short-circuits at first falsy operand)
- `0 or 5` → `5` (returns first truthy operand)
- `5 or 7` → `5` (returns first truthy operand)

Falsy values in Python: `0`, `0.0`, `''`, `[]`, `()`, `{}`, `None`, `False`. Everything else is truthy.

Exam pitfalls.

- `not x` always returns a bool. `x or y` does not.
- `if []:` is False. `if [0]:` is True. (Empty list vs list containing 0.)

---

2. Higher-Order Functions and Lambda

[Slides 04, 06]

Higher-Order Function (HOF)

Definition. A function that takes another function as an argument or returns a function as a result.

Plain English. Functions are first-class values in Python — you can pass them around like numbers or strings.

Canonical templates.

```
# HOF taking a function
def apply_twice(f, x):
    return f(f(x))

# HOF returning a function (closure)
def make_adder(n):
    def adder(k):
        return k + n        # `n` from enclosing frame
    return adder            # return the function object (no parens!)

add5 = make_adder(5)        # add5 is a function
add5(3)                     # → 8
```

Lambda

Definition. An anonymous function expression with a single expression body.

Plain English. Quick way to write short, unnamed functions. Cannot contain statements (no `if`/`return`/loops as statements).

Canonical template.

```
square = lambda x: x * x        # lambda x: <expression>
adder  = lambda a, b: a + b
```

Equivalent to:

```
def square(x):
    return x * x
```

Differences from `def`.

- Lambda is an *expression*, returns the function value. `def` is a *statement*, binds a name.
- Lambda has no intrinsic name (`__name__` is `'<lambda>'`).
- Body of lambda is a single expression. No multi-line statements.

Exam patterns / pitfalls.

- "Convert this `def` to a lambda" → only works if the body is `return <expr>`.
- "Predict what `(lambda x: x * x)(3)` evaluates to" → call lambda inline → `9`.
- Closures capture variables by *reference*, not by value. The classic trap:

```
funcs = [lambda: i for i in range(3)]
[f() for f in funcs]    # → [2, 2, 2], not [0, 1, 2]
```

---

3. The Environment Model and Tracing

[Slide 05]

Frames and Environments

Definition. A *frame* is a mapping from names to values. An *environment* is a sequence of frames (current, parent, ..., global).

Plain English. When Python looks up a name, it searches frames from innermost to outermost. The order is determined by the function's parent frame at definition time, not at call time.

The four rules of the environment model.

1. **`def f(x): body`** creates a function value `func f(x) [parent=<current frame>]` and binds `f` in the current frame.
2. **`f(arg)`** opens a new frame called `f`. The new frame's *parent* is the function's recorded parent frame (NOT the caller's frame). Bind formal params to argument values; execute body.
3. **Name lookup** walks from current frame to parent to grandparent ... to global. First match wins.
4. **Return value** is the result of the function call expression. It is not bound in the new frame.

Tracing template (drawing an environment diagram)

```
make_adder(3) called:
  → f1: make_adder          [parent=Global]
        n=3
  → def adder(k):  creates  func adder [parent=f1]
  → returns func adder

add5 = make_adder(3)
add5(4) called:
  → f2: adder               [parent=f1]
        k=4
  → looks up `n`:
        not in f2 → walks to f1 (parent) → finds n=3
  → returns 4 + 3 = 7
```

Exam patterns / pitfalls

- The new frame's parent is the **function's parent frame at definition time**, not the calling frame. This is the #1 source of wrong answers.
- A `def` inside a `def` creates a function whose parent is the enclosing function's frame at the time of definition. Each call of the outer creates a new such frame, so the inner function can have a different parent each call.

- Lambdas have parents too — same rule.
- A `return` statement does not modify any frame; it just hands the value back to the caller.
- Reassigning a name (`x = x + 1`) inside a function modifies the *current* frame's binding, not the parent's.

---

4. Recursion and Tree Recursion

[Slides 09–10]

Linear Recursion

Definition. A function that calls itself once per call. Each call reduces the problem.

Plain English. Trust the recursive call: assume it works on the smaller problem. Combine its result with the current step.

Canonical template.

```
def f(n):
    if <base case>:
        return <base value>
    return <combine using f(smaller(n))>

# Example: factorial
def fact(n):
    if n == 0:
        return 1
    return n * fact(n - 1)
```

Tree Recursion

Definition. A function that calls itself more than once per call. Branching call structure.

Plain English. Used when each problem splits into multiple sub-problems. Time complexity is exponential without memoization.

Canonical examples.

```
def fib(n):
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)

def count_partitions(n, m):
    if n == 0:
        return 1
    if n < 0 or m == 0:
        return 0
    return count_partitions(n - m, m) + count_partitions(n, m - 1)
```

Mutual Recursion

Definition. Two functions call each other to solve a problem.

Plain English. Can always be rewritten as single recursion, but mutual recursion is sometimes cleaner.

```
def is_even(n):
    if n == 0: return True
    return is_odd(n - 1)

def is_odd(n):
    if n == 0: return False
    return is_even(n - 1)
```

Recursion design checklist

1. Identify the base case(s). Make sure they cover all "smallest" inputs.
2. Identify the recursive case. Make sure it makes progress toward a base case.
3. Trust the recursive call (don't trace it in your head).
4. Combine the recursive result with the current step.

Exam patterns / pitfalls

- Forgetting a base case → infinite recursion → `RecursionError`.
- Recursive case not making progress (`f(n)` calls `f(n)`).
- Off-by-one: "do something for `i = 0..n`" often becomes `f(0)` base, recurse on `n-1`.
- Tree recursion problems often have multiple base cases — list all of them carefully.

---

5. Sequences, Containers, and Comprehensions

[Slides 11–12]

Lists, Tuples, Sets, Dicts

Container	Mutable?	Ordered?	Allows duplicates?	Lookup time
`list`	Yes	Yes	Yes	O(n) by value
`tuple`	No	Yes	Yes	O(n) by value
`set`	Yes	No	No	O(1) by value
`dict`	Yes	Insertion order (Py3.7+)	Keys unique	O(1) by key

Slicing

Template. `seq[start:stop:step]`

- Negative `start`/`stop` count from the end.
- `seq[::-1]` reverses.
- `seq[:]` returns a *shallow copy*.

List Comprehensions

Template. `[<expr> for <var> in <iter> if <cond>]`

Plain English. Build a new list by iterating, optionally filtering.

Examples.

```
[x*x for x in range(5)]                    # [0, 1, 4, 9, 16]
[x for x in range(10) if x % 2 == 0]       # [0, 2, 4, 6, 8]
[(i, j) for i in range(3) for j in range(3) if i < j]   # [(0,1), (0,2), (1,2)]
```

Common HOFs over sequences

```
sum([1, 2, 3])                # 6
max([1, 2, 3])                # 3
min([1, 2, 3])                # 1
list(map(square, [1, 2, 3]))  # [1, 4, 9]
list(filter(lambda x: x > 1, [1, 2, 3]))  # [2, 3]
```

6. Mutability, Aliasing, and Identity

[Slide 15]

`is` vs `==`

Definition. `==` compares value. `is` compares object identity (same object in memory).

Plain English. Two lists `[1, 2, 3]` can have equal contents (`==` is `True`) but be distinct objects (`is` is `False`).

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a
a == b    # True (same value)
a is b    # False (different objects)
a is c    # True (same object, c is an alias)
```

Canonical use. Always use `is` for comparison with `None`: `if x is None:` (idiomatic).

Aliasing

Definition. When two names refer to the same mutable object.

Plain English. Mutating through one name affects all aliases. The most common bug source in beginner Python.

```
a = [1, 2, 3]
b = a               # b aliases a — same list
b.append(4)
print(a)            # [1, 2, 3, 4] — surprise!
```

Mutating vs Reassigning

- **Mutating:** `a.append(4)`, `a[0] = 99`, `a.extend(...)` — modifies the existing object.
- **Reassigning:** `a = [99, 2, 3]` — points `a` at a new object; old object unchanged.
- An alias `b = a` is broken by reassigning `a`, but mutating `a` still affects `b`.

Exam patterns / pitfalls

- "Predict the output" of code that mixes mutation and aliasing → always draw the box-and-arrow diagram.
- Function arguments: lists are passed by reference. Mutating a list inside a function affects the caller.
- `x = x + [1]` creates a new list (reassignment); `x.append(1)` mutates in place.

7. Iterators and Generators

[Slides 16–17]

Iterator Protocol

Definition. An iterator is any object with a `__next__` method (returning the next value or raising `StopIteration`). An iterable has `__iter__` (returning a new iterator).

Plain English. `for x in obj:` calls `iter(obj)` to get an iterator, then calls `next()` until `StopIteration`.

```
it = iter([1, 2, 3])
next(it)      # 1
next(it)      # 2
next(it)      # 3
next(it)      # raises StopIteration
```

Generators

Definition. A function with `yield` instead of `return`. Calling it returns a generator object (an iterator). Each `next()` advances the function until the next `yield`.

Plain English. `yield` *pauses* the function and remembers where it left off. Resuming continues from there.

Canonical template.

```
def countdown(n):
    while n > 0:
        yield n
        n -= 1

g = countdown(3)
next(g)      # 3
next(g)      # 2
next(g)      # 1
next(g)      # raises StopIteration

# Use in for loop:
for x in countdown(3):
    print(x)        # prints 3, 2, 1
```

Generator with `yield from`

Plain English. Delegates iteration to another iterable. Useful for tree-traversal generators.

```
def all_pairs(lst):
    for i in range(len(lst)):
        for j in range(i+1, len(lst)):
            yield lst[i], lst[j]

def flatten(nested):
    for item in nested:
        if isinstance(item, list):
            yield from flatten(item)        # recurse
        else:
            yield item
```

Exam patterns / pitfalls

- Generators are *lazy*. `g = countdown(3)` does NOT run the function body. `next(g)` starts it.
- A generator can only be iterated once. After exhaustion, calling `next()` raises `StopIteration` forever.
- "Implement a generator that yields the powers of 2 less than n" — classic short problem.

---

8. Object-Oriented Programming

[Slides 18–23]

Class Definition

Canonical template.

```
class Account:
    interest = 0.02             # CLASS attribute (shared by all instances)

    def __init__(self, holder):  # constructor: called by Account('Alice')
        self.holder = holder     # INSTANCE attribute
        self.balance = 0

    def deposit(self, amount):
        self.balance += amount
        return self.balance
```

Attribute Lookup Rule

Definition. Look up `obj.attr` first in the *instance dictionary*, then in the *class dictionary*, then up the inheritance chain.

Plain English. Each instance has its own attributes. The class also has attributes. If you ask `acc.interest`, Python checks `acc`'s instance dict — not there — checks `Account`'s class dict — finds `0.02`.

Important. `acc.interest = 0.05` creates an *instance* attribute that shadows the class attribute, only for `acc`. Other instances still see `0.02`.

Method Calls

Definition. `acc.deposit(100)` desugars to `Account.deposit(acc, 100)`. The instance is passed as `self`.

Plain English. Methods are just functions stored on the class. The dot operator binds `self`.

Inheritance

Canonical template.

```
class CheckingAccount(Account):
    interest = 0.01             # override class attribute
    withdraw_fee = 1

    def withdraw(self, amount):
```

```
        return Account.withdraw(self, amount + self.withdraw_fee)
        # or: super().withdraw(amount + self.withdraw_fee)
```

Method resolution order (MRO). Subclass first, then parent, then grandparent. Python uses C3 linearization for multiple inheritance.

Special Methods (dunder methods)

Method	When called
`__init__(self, ...)`	construction
`__repr__(self)`	`repr(obj)`, debugger display, REPL
`__str__(self)`	`str(obj)`,`print(obj)`
`__eq__(self, other)`	`self == other`
`__add__(self, other)`	`self + other`
`__len__(self)`	`len(obj)`
`__iter__(self)`	`iter(obj)`, makes class iterable
`__getitem__(self, k)`	`obj[k]`

Exam patterns / pitfalls

- "Find the class attribute" vs "find the instance attribute" — always check the instance dict first.
- `__init__` is NOT the constructor in the strict sense — it's the initializer. Construction happens before `__init__` runs.
- `super().method(...)` vs `Parent.method(self, ...)` — both work; `super()` is cleaner.
- Class attribute mutation: `Account.interest = 0.05` changes the value for all instances that haven't shadowed it.

9. Linked Lists (`Link` class)

[Slides 14, 22]

The `Link` class — memorize this exactly

```
class Link:
    empty = ()                                  # sentinel for end of list

    def __init__(self, first, rest=empty):
        assert rest is Link.empty or isinstance(rest, Link)
        self.first = first
        self.rest = rest                        # always a Link or Link.empty
```

Names to know

Name	What it is
`lnk.first`	value at this node
`lnk.rest`	the next `Link` (or `Link.empty`)
`Link.empty`	sentinel — the empty tuple `()`

Base case check (use `is`, not `==`)

```
if lnk is Link.empty:
    ...
```

Core templates

Traverse / read:

```
def length(lnk):
    if lnk is Link.empty:
        return 0
    return 1 + length(lnk.rest)
```

Filter (return new list excluding some elements):

```
def exclude(lnk, x):
    if lnk is Link.empty:
        return Link.empty
    if lnk.first == x:
        return exclude(lnk.rest, x)
```

```
    return Link(lnk.first, exclude(lnk.rest, x))
```

Build (prepend, iterative):

```
def from_list(lst):
    result = Link.empty
    for x in reversed(lst):
        result = Link(x, result)
    return result
```

Mutate in place (set the second value to 99):

```
lnk.rest.first = 99
```

Insert at end (mutating):

```
def append_last(lnk, value):
    while lnk.rest is not Link.empty:
        lnk = lnk.rest
    lnk.rest = Link(value)
```

Exam patterns / pitfalls

- `Link.empty` is `()` — comparing with `==` works but `is` is the idiomatic 61A check.
- Recursive `Link` functions almost always have the same shape: base case on `Link.empty`, recursive case combines `lnk.first` with the result of `... lnk.rest`.
- Building the result in reverse order: a common bug. Either build with `reversed()`, or use direct recursion (which preserves order).

---

10. Trees (`Tree` class)

[Slide 14]

The `Tree` class — memorize this exactly

```
class Tree:
    def __init__(self, label, branches=[]):
        for b in branches:
            assert isinstance(b, Tree)
        self.label = label
        self.branches = list(branches)

    def is_leaf(self):
        return not self.branches
```

Names to know

- `t.label` — value at this node
- `t.branches` — list of subtrees (each a `Tree`)
- `t.is_leaf()` — `True` iff no branches

Core templates

Sum all labels:

```
def total(t):
    return t.label + sum(total(b) for b in t.branches)
```

Count leaves:

```
def count_leaves(t):
    if t.is_leaf():
        return 1
    return sum(count_leaves(b) for b in t.branches)
```

Map a function over labels (returns new tree):

```
def tree_map(t, f):
    return Tree(f(t.label), [tree_map(b, f) for b in t.branches])
```

All paths from root to leaf:

```
def paths(t):
    if t.is_leaf():
        return [[t.label]]
    result = []
    for b in t.branches:
```

```
        for path in paths(b):
            result.append([t.label] + path)
    return result
```

Exam patterns / pitfalls

- Forgetting `t.is_leaf()` base case → wrong recursion structure for empty branches list.
- Mutating shared default arguments. Default `branches=[]` is shared across calls; always copy with `list(branches)` in `__init__`.
- The pattern `[f(b) for b in t.branches]` recurses on each branch.

---

11. Scheme — Syntax and Semantics

[Slide 26]

Atom vs Pair

Definition. Scheme programs and data are built from atoms (numbers, booleans, symbols, `()`) and *pairs* `(cons a b)`.

Plain English. Lists in Scheme are nested pairs ending in `nil`/`'()`. `(1 2 3)` is `(cons 1 (cons 2 (cons 3 '())))`.

Special Forms (memorize)

```
(define x 5)                          ; bind x to 5

(define (square x) (* x x))           ; define a function

(lambda (x) (* x x))                  ; anonymous function

(if predicate then-expr else-expr)    ; if expression

(cond ((test1) result1)               ; multi-branch cond
      ((test2) result2)
      (else default))

(let ((x 1) (y 2)) body)              ; local bindings

(and e1 e2 ...)                       ; short-circuit and
(or e1 e2 ...)                        ; short-circuit or

(begin e1 e2 ... eN)                  ; sequence; returns last
```

Pair / List Operations

```
(cons 1 2)                ; → (1 . 2)
(cons 1 (cons 2 '()))     ; → (1 2)
(car '(1 2 3))            ; → 1
(cdr '(1 2 3))            ; → (2 3)
(null? '())               ; → #t
(pair? (cons 1 2))        ; → #t
(list 1 2 3)              ; → (1 2 3)
(append '(1 2) '(3 4))    ; → (1 2 3 4)
```

Mutation (61A's mutable pairs)

```
(set-car! p new-first)    ; mutate first
(set-cdr! p new-rest)     ; mutate rest
```

Canonical Scheme functions

```
(define (length lst)
  (if (null? lst)
      0
      (+ 1 (length (cdr lst)))))

(define (factorial n)
  (if (= n 0)
      1
      (* n (factorial (- n 1)))))

(define (count-leaves t)
  (cond ((null? t) 0)
        ((not (pair? t)) 1)
        (else (+ (count-leaves (car t))
                 (count-leaves (cdr t))))))

(define (map f lst)
  (if (null? lst)
      '()
      (cons (f (car lst)) (map f (cdr lst)))))
```

Exam patterns / pitfalls

- `cons` makes a pair. A *list* is just a chain of pairs ending in `'()`.
- `(cons 1 (cons 2 (cons 3 '())))` displays as `(1 2 3)`. `(cons 1 (cons 2 3))` displays as `(1 2 . 3)` — the dot indicates an "improper list."
- `'()` is the empty list. `(null? '())` → `#t`.
- Forgetting parens (or having extra ones) is the #1 bug source.
- `(define f 5)` creates a binding; `(define (f x) (* x x))` creates a function.

12. Tail Calls

[Likely Slide 24 — verify in cs61a.org Composing Programs Ch 3.5 or equivalent]

Tail Call

Definition. A function call is in *tail position* if its return value is the return value of the enclosing function — no further computation happens after it returns.

Plain English. `(define (f x) (g x))` — the call to `g` is a tail call. `(define (f x) (+ 1 (g x)))` — NOT a tail call (we add 1 after).

Why it matters. Scheme implementations *must* perform tail-call optimization (TCO): tail calls don't add to the call stack. So tail-recursive functions run in constant space. Python does NOT do TCO.

Tail position rules

A call `(f ...)` is in tail position if:

- It's the body of a `lambda` or `define`-function.
- It's the last expression in a `begin`.
- It's a branch result of `if`, `cond`, `case`, `when`, `unless`.
- It's the body of a `let` (not the bindings).

Converting non-tail to tail recursive

Pattern: accumulator. Add a parameter that accumulates the result.

```
;; NOT tail recursive (the multiplication waits for the recursive call):
(define (factorial n)
  (if (= n 0)
      1
      (* n (factorial (- n 1)))))

;; Tail recursive (uses accumulator):
(define (factorial n)
  (define (helper n acc)
    (if (= n 0)
        acc
        (helper (- n 1) (* n acc))))
  (helper n 1))
```

Exam patterns / pitfalls

- "Is this tail recursive?" → look at the very last operation. If it's an arithmetic op or `cons` *after* the recursive call, NOT tail recursive.

- "Convert this to tail recursive" → introduce an accumulator parameter (helper function pattern).
- A `cond` branch's last expression is in tail position only if the `cond` itself is in tail position.

---

13. Streams

[Likely Slide 25 — verify in cs61a.org Composing Programs Ch 4.2 or equivalent]

Stream

Definition. A lazy list — a pair where the *cdr* is a thunk (no-arg lambda) that produces the rest only when forced.

Plain English. Lets you build infinite sequences. The first element is computed; the rest is a promise.

`cons-stream` and stream basics (61A convention)

```
(cons-stream <first> <rest>)       ; first is evaluated; rest is delayed (not yet evaluated)
(car s)                            ; first element (evaluated)
(cdr-stream s)  or  (cdr s)        ; force the rest — depending on 61A's exact API
(null? s)                          ; check empty stream (often '() / nil-stream)
```

Note on API. Different 61A semesters use slightly different stream APIs. Common functions are `stream-car`, `stream-cdr`, `stream-ref`, `stream-map`, `stream-filter`. The semantics are the same; only the names differ. Verify by re-reading your project Scheme code.

Canonical stream definitions

```
;; Infinite stream of integers starting at n
(define (integers-from n)
  (cons-stream n (integers-from (+ n 1))))

;; Map over a stream
(define (stream-map f s)
  (if (null? s)
      '()
      (cons-stream (f (car s)) (stream-map f (cdr-stream s)))))

;; Filter a stream
(define (stream-filter pred s)
  (cond ((null? s) '())
        ((pred (car s))
         (cons-stream (car s) (stream-filter pred (cdr-stream s))))
        (else (stream-filter pred (cdr-stream s)))))

;; Take first n elements as a list
```

```
(define (stream-take s n)
  (if (or (null? s) (= n 0))
      '()
      (cons (car s) (stream-take (cdr-stream s) (- n 1)))))
```

Exam patterns / pitfalls

- Stream functions look like list functions but use `cons-stream` instead of `cons` so the recursive part isn't evaluated immediately.
- Forcing the cdr too eagerly defeats the purpose of streams (and may cause infinite loop for infinite streams).
- "Show what `(stream-take (integers-from 0) 5)` returns" → `(0 1 2 3 4)`.

---

14. Interpreters — Calculator and Scheme

[Slides 28–29]

Read-Eval-Print Loop (REPL)

Plain English. An interpreter has three phases:

1. **Read** — parse user input string into an internal representation (typically nested Pairs in 61A).
2. **Eval** — evaluate the parsed expression in some environment.
3. **Print** — display the result.

Calculator language (Slide 28)

Definition. A small subset of Scheme: only `+`, `-`, `*`, `/` with numeric arguments. No variables, no functions.

Plain English. The 61A toy language. Easier than full Scheme. Lets you understand the eval/apply cycle without OOP/closures complexity.

Eval rules (Calculator).

1. A number evaluates to itself.
2. A call expression `(op operand1 operand2 ...)`:

- Evaluate each operand recursively.
- Apply the operator to the resulting values.

Implementation skeleton (Python).

```
def calc_eval(expr):
    if isinstance(expr, (int, float)):
        return expr                                          # numbers
evaluate to themselves
    op, *args = expr                                         # destructure
    arg_values = [calc_eval(a) for a in args]                # eager arg
evaluation
    return calc_apply(op, arg_values)


def calc_apply(op, args):
    if op == '+': return sum(args)
    if op == '-': return args[0] - sum(args[1:]) if len(args) > 1 else -args[0]
    if op == '*':
        result = 1
        for a in args: result *= a
        return result
    if op == '/': return args[0] / args[1]
    raise ValueError(f'unknown op: {op}')
```

Full Scheme interpreter (Slide 29 — the project)

Plain English. Full Scheme adds: variables, `define`, `lambda`, special forms (`if`, `cond`, `let`, `quote`, `and`, `or`), procedure calls. The eval function must distinguish *special forms* from *regular procedure calls* before evaluating arguments.

`scheme_eval(expr, env)` — high-level skeleton.

```
def scheme_eval(expr, env):
    # 1. Self-evaluating: numbers, strings, booleans
    if self_evaluating(expr):
        return expr
    # 2. Symbol: look up in env
    if isinstance(expr, str):
        return env.lookup(expr)
    # 3. Pair: it's a call expression OR a special form
    first, rest = expr.first, expr.rest
    if first in SPECIAL_FORMS:
        return SPECIAL_FORMS[first](rest, env)        # don't eval args yet
    # 4. Procedure call: eval operator and operands, then apply
    procedure = scheme_eval(first, env)
    args = [scheme_eval(operand, env) for operand in rest_to_list(rest)]
    return scheme_apply(procedure, args, env)
```

`scheme_apply(procedure, args, env)` — high-level skeleton.

```
def scheme_apply(procedure, args, env):
    if isinstance(procedure, BuiltinProcedure):
        return procedure.fn(*args)
    if isinstance(procedure, LambdaProcedure):
        new_env = Environment(parent=procedure.env)             # parent =
lambda's defining env
        for param, arg in zip(procedure.params, args):
            new_env.bind(param, arg)
        return scheme_eval(procedure.body, new_env)
    raise SchemeError('not a procedure')
```

Special forms vs procedure calls

Critical distinction.

- **Procedure call:** evaluate all operands first, then apply the procedure. (`(+ 1 2)` → eval `1`, eval `2`, then call `+`)
- **Special form:** the special-form handler decides what (if anything) to evaluate. (`(if #t 1 (error))` — only eval the chosen branch; don't eval `(error)`!)

This is why `if`, `define`, `lambda`, `cond`, `quote`, `and`, `or`, `let` need their own handlers — they violate the "always eval all args" rule.

Exam patterns / pitfalls

- "Modify Calculator to support `pow`" — extend `calc_apply` and (maybe) `calc_eval` for variadic ops.
- "Add a special form `repeat`" — write a handler that takes the unevaluated form and evaluates parts manually.
- "Why is `if` a special form?" — because if it were a procedure, both branches would be evaluated, defeating its purpose.
- The lambda's environment captures the *defining* environment, not the *calling* one — same rule as Python.

---

15. Programs as Data

[Slide 30]

Code is Data

Definition. In Scheme, programs are just nested pairs (the same data structure as lists). Therefore, you can manipulate them with normal pair operations.

Plain English. The expression `(+ 1 2)` is a *pair* whose first element is the symbol `+` and whose rest is the list `(1 2)`. You can take it apart with `car` and `cdr`, transform it, and pass the result to `eval` to run it.

quote

Definition. `(quote x)` (or `'x`) returns `x` literally — without evaluating it.

Plain English. Quote turns "code" into "data." `(+ 1 2)` evaluates to `3`. `'(+ 1 2)` evaluates to the *list* `(+ 1 2)` — the symbol `+`, the number `1`, the number `2`.

```
(+ 1 2)           ; → 3 (evaluated)
'(+ 1 2)          ; → (+ 1 2) (a list of 3 elements)
(car '(+ 1 2))    ; → +  (the symbol)
(cdr '(+ 1 2))    ; → (1 2)
```

Code-manipulation example

```
;; Take an expression like (+ 1 2) and return (* 1 2)
(define (plus-to-times expr)
  (cons '* (cdr expr)))

(plus-to-times '(+ 1 2))   ; → (* 1 2)

;; Then evaluate it:
(eval (plus-to-times '(+ 1 2)) (the-environment))   ; → 2
```

Exam patterns / pitfalls

- "Write a function that swaps the first two arguments of any function call expression."
- "Why is `'foo` different from `foo`?" — `foo` evaluates the symbol; `'foo` returns the symbol itself.
- Programs as data is the foundation for macros — see next section.

16. Macros

[Slide 31]

Macro

Definition. A macro is a code transformation: it takes the *unevaluated* code as input, produces new code, which is then evaluated.

Plain English. Functions take values. Macros take code. The macro substitutes parameters into a template and the result is evaluated.

`define-macro` (61A's macro form)

Canonical template.

```
(define-macro (name params)
  template-that-returns-an-expression)
```

The macro receives unevaluated arguments. The result of the macro is a Scheme expression, which Scheme then evaluates in the call site's environment.

Example: `unless`

```
;; unless executes the body only if the condition is FALSE
(define-macro (unless condition body)
  (list 'if condition '() body))
;; → (unless (= x 0) (/ 1 x)) expands to (if (= x 0) () (/ 1 x))
```

Quasiquote (helps build templates)

```
`(if ,condition () ,body)     ; backtick = quasiquote, comma = unquote
;; equivalent to:
(list 'if condition '() body)
```

Exam patterns / pitfalls

- A macro is NOT a function. Trying to use a macro as a function fails because functions evaluate args before the call.
- "Write a macro `swap-args` that swaps the args of any 2-arg call." → use list construction (or quasiquote).
- "When does the macro body run?" — at *expansion* time, before evaluation. The macro's job is to produce code; that code runs later.
- Hygiene: 61A macros are NOT hygienic — variable name capture can cause bugs. Watch for accidental shadowing.

---

17. SQL — Queries

[Slides 32–33]

Basic SELECT

Canonical template.

```
SELECT column1, column2
  FROM table
 WHERE condition
 ORDER BY column [ASC|DESC]
 LIMIT n;
```

Plain English. Pick columns from rows of a table where a condition holds, optionally sorted and limited.

Renaming and computed columns

```
SELECT name AS person, age * 12 AS age_in_months
  FROM people;
```

Joins

```
-- Cross product (Cartesian product, then filter):
SELECT a.name, b.name
  FROM people AS a, people AS b
 WHERE a.parent = b.id;

-- Equivalent INNER JOIN:
SELECT a.name, b.name
  FROM people AS a
       JOIN people AS b ON a.parent = b.id;
```

Plain English. Joining two tables produces all combinations of rows; the `WHERE` (or `ON`) clause keeps only the meaningful ones.

Self-join (parent/child relationship)

```
SELECT child.name, parent.name
  FROM dogs AS child
       JOIN dogs AS parent ON child.parent_id = parent.id;
```

Exam patterns / pitfalls

- Forgetting to alias when joining a table to itself → ambiguous column names.
- `WHERE` filters rows BEFORE grouping. `HAVING` filters AFTER aggregation.
- Cross-product without a `WHERE`/`ON` clause produces N×M rows — almost never what you want.

18. SQL — Aggregation and Recursive CTEs

[Slides 34–35]

GROUP BY + Aggregate functions

Canonical template.

```
SELECT department, COUNT(*) AS num_employees, AVG(salary) AS avg_salary
  FROM employees
 GROUP BY department
HAVING COUNT(*) >= 5;
```

Aggregate functions. `COUNT(*)`, `COUNT(col)`, `SUM(col)`, `AVG(col)`, `MIN(col)`, `MAX(col)`.

Plain English. `GROUP BY` collapses rows that share a value in the grouping column into one row. Aggregates produce one number per group.

Rules

- Every column in `SELECT` that isn't an aggregate must appear in `GROUP BY`.
- `WHERE` filters before grouping. `HAVING` filters after.

Recursive CTE (Common Table Expression)

Canonical template.

```
WITH RECURSIVE name(col1, col2, ...) AS (
    -- Base case
    SELECT ... FROM ...
    UNION
    -- Recursive case (refers to `name`)
    SELECT ... FROM name JOIN ... ON ... WHERE ...
)
SELECT * FROM name;
```

Plain English. A self-referential temporary table. Used for hierarchical or iterative queries: ancestors, paths, generating sequences.

Example: ancestors (transitive closure)

```
WITH RECURSIVE ancestor(name, ancestor) AS (
    SELECT child, parent FROM family
    UNION
    SELECT a.name, f.parent
      FROM ancestor AS a
           JOIN family AS f ON a.ancestor = f.child
```

```
)
SELECT * FROM ancestor;
```

Example: integers 1 to 10

```
WITH RECURSIVE nums(n) AS (
    SELECT 1
    UNION
    SELECT n + 1 FROM nums WHERE n < 10
)
SELECT * FROM nums;
```

Exam patterns / pitfalls

- Forgetting `UNION` between base and recursive cases.
- Recursive case must reference the CTE; otherwise it's not actually recursive.
- Termination: recursive case must eventually return zero rows (or a `WHERE` clause must cut it off).
- Column names declared after the CTE name (`WITH RECURSIVE name(col1, col2)`) — column count must match the SELECT.

Closing notes

This guide is the structural catalog. Use it together with:

- `61A_Exam_Reference.md` — your pattern-trigger reference (still your primary).
- `MT2_Code_Trigger_Sheet.md` — Linked List + Tree + OOP trigger sheet.
- Past exam: `61a-fa25-final.pdf` (or sp25/su25 final) — take timed.
- The `cs61a/` codebase — your own project work, especially the Scheme project.

Verify before exam day:

- Final exam exact time and location at https://cs61a.org/articles/about-61a/
- Cheat sheet allowance for SP26 (typically 2 double-sided handwritten pages).
- Whether Streams (Slide 25) and Tail Calls (Slide 24) are in scope — verify on cs61a.org schedule. If yes, drill them.

Memorize cold (cheat sheet candidates):

- `Link` class definition + `Link.empty` sentinel
- `Tree` class definition + `is_leaf()` method
- Environment-model rule: new frame's parent = function's parent at definition
- Scheme special forms list (`define`, `lambda`, `if`, `cond`, `let`, `and`, `or`, `quote`, `begin`)
- `cons`/`car`/`cdr` and how lists are built
- Tail position rules (last in body / branch of if/cond / last in begin)
- `cons-stream` semantics (first eager, rest delayed)
- `scheme_eval` skeleton (self-eval, symbol, special form, procedure call)
- SQL: `SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY ... LIMIT`
- Recursive CTE: `WITH RECURSIVE name(cols) AS (base UNION recursive)`